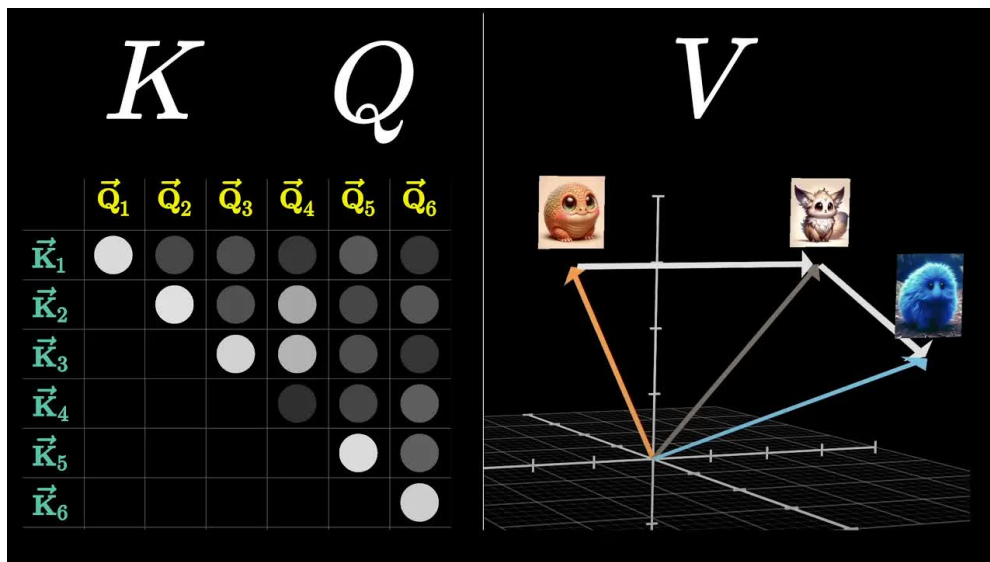


Attention 机制详解：从直觉到矩阵运算

Csurfing's Agent Lab

2026 年 7 月 30 日



视频作者/频道: 3Blue1Brown

发布日期: 2024-04-07

视频时长: 26:09

视频链接: <https://www.youtube.com/watch?v=eMlx5fFNoYc>

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

目录

1 引言：从词嵌入到上下文理解	2
1.1 Transformer 结构速览	2
1.2 Attention 要解决的问题	2
1.3 本章小结	3
2 位置编码：给 Token 加上序号	3
2.1 为什么需要位置编码	3
2.2 本章小结	4
3 注意力机制的核心直觉	4
3.1 从“查资料”的类比理解 Q、K、V	4
3.2 打分的数学直觉	4
3.3 从相似度到权重：Softmax 的作用	5
3.4 加权求和：最终输出	6
3.5 本章小结	6
4 Query、Key、Value 矩阵：可学习的参数	6
4.1 从向量到矩阵	6
4.2 完整的 Self-Attention 公式	7
4.3 本章小结	8
5 Self-Attention 如何在 Transformer 中流动	8
5.1 单层 Self-Attention 的完整路径	8
5.2 本章小结	9
6 多头注意力：并行关注不同方面	9
6.1 为什么需要多头？	9
6.2 参数规模	10
6.3 本章小结	10
7 总结与延伸	11
7.1 核心机制回顾	11
7.2 视频中的关键洞察	11
7.3 拓展阅读	11

1 引言：从词嵌入到上下文理解

1.1 Transformer 结构速览

3Blue1Brown 的本章视频紧接前一章关于 Transformer 整体架构的内容，聚焦于其最核心的组件——**注意力机制 (Attention Mechanism)**。这一机制首次出现在 2017 年那篇划时代的论文《Attention Is All You Need》中。

在深入注意力机制的数学细节之前，我们需要先建立一个直观：**注意力机制要解决的根本问题，是让模型理解上下文中的词义。**

预备知识：词嵌入 (Embedding)

前一章的核心结论：输入文本被切分成 token (词元)，每个 token 被映射为一个高维向量，称为嵌入向量 (embedding vector)。这些向量所在的语义空间中，**方向对应语义**——沿某个方向移动可以从一个词移动到另一个语义相近的词。例如，添加一个特定的向量可以把”国王”的嵌入变成”王后”的嵌入。

1.2 Attention 要解决的问题

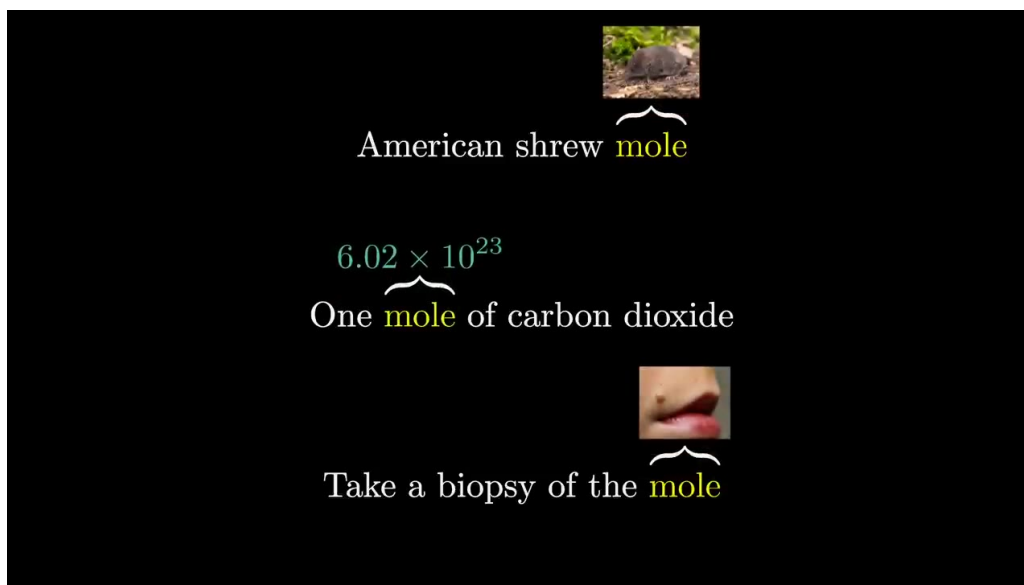


图 1: 相同的词在不同语境中有不同含义——Attention 要解决的核心问题¹

考虑三个短语：

- *American shrew mole* (美洲鼩鼱)
- *one mole of carbon dioxide* (一摩尔二氧化碳)
- *take a biopsy of the mole* (对痣做活检)

人眼一看就知道，”mole”在三句话里分别是动物、化学单位和皮肤病。但 Transformer 的第一步仅仅是查表 (lookup table) —— 把每个 token 映射到一个固定向量。不管上下文如何，同样的词得到同样的初始嵌入向量。

¹ 视频画面时间区间：00:02:10–00:02:30

Attention 的核心思想

Attention 的答案是：让每个 token 主动”查看”其他所有 token，根据相关性从它们那里聚合信息。这样，”mole” 的嵌入就能从周围的词（”shrew”、”carbon dioxide”、”biopsy”）获取足够的信息来区分自己当前的含义。

1.3 本章小结

本章建立了问题框架：初始嵌入不含上下文信息，Attention 机制就是为解决这一缺陷而设计的。后续各节将逐步拆解其运作细节。

2 位置编码：给 Token 加上序号

2.1 为什么需要位置编码

Attention 机制有一种被忽视的特性：它本身不感知顺序。如果把句子中的词打乱顺序，注意力打分的结果完全不变——因为它只是两两计算相似度。

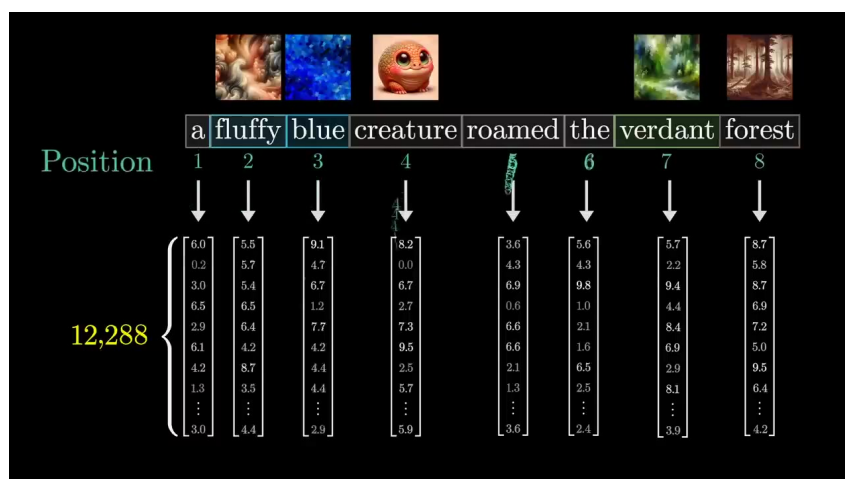


图 2: 位置编码给每个 token 添加位置信息，使模型能感知词的先后顺序²

位置编码（Position Encoding）的解决办法是：在初始嵌入向量上加上一个与位置相关的信号。这样，模型同时看到了”内容（嵌入）”和”位置（编码）”，就能区分”猫追老鼠”和”老鼠追猫”。

位置编码的直觉

位置编码不是简单的 1, 2, 3, ... 序号。Transformer 使用正弦和余弦函数的组合来编码位置，这样可以：

- 产生连续的、有界的值（方便训练）
- 让模型容易学会”相对位置”（第 5 个和第 6 个之间的偏移量）
- 能泛化到训练时没见过的更长的序列

²视频画面时间区间：00:05:10–00:05:30

2.2 本章小结

位置编码是 Transformer 的基础步骤：词嵌入 + 位置信息 $\rightarrow$ 带位置上下文的输入向量。之后的注意力计算都在这个加和后的向量上进行。

3 注意力机制的核心直觉

3.1 从”查资料”的类比理解 Q、K、V

Attention 的运作方式可以类比为智能的图书馆检索系统：

- **Query (查询)**：”我要找什么”——类似于你在搜索引擎输入的关键词
- **Key (键)**：”我是什么内容，供别人搜索”——类似于每本书的标题和标签
- **Value (值)**：”如果你选中我，我给你什么信息”——类似于书的内容本身

3.2 打分的数学直觉

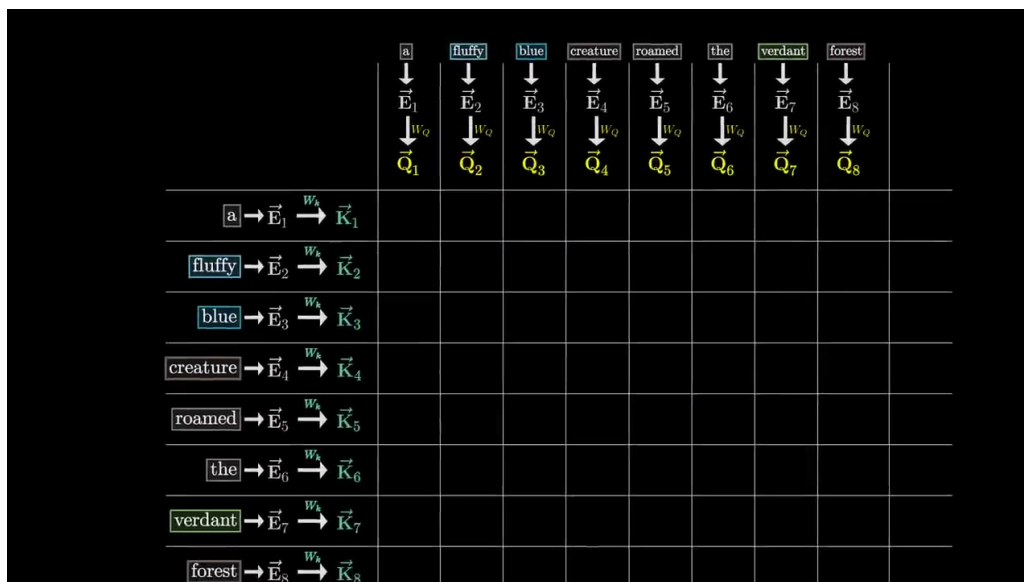


图 3: 每个 token 的 Query 与所有 Key 计算相似度 (affinity scores)³

关注当前 token (比如句子中正在处理的”mole”), 它的 Query 向量会去”匹配”句子中所有其他 token 的 Key 向量。匹配程度 (相似度得分) 可以通过向量点积来衡量：

- 点积越大 $\rightarrow$ Query 和 Key 越”对齐” $\rightarrow$ 这两个 token 的相关性越高
- 点积越小 $\rightarrow$ Query 和 Key 不太相关 $\rightarrow$ 这两个 token 的关联度低

关键洞察：全连接而非硬选择

Attention 的独特之处在于：它不是”选一个”，而是”把所有信息按权重混合”。每个 token 都会把注意力分配到所有其他 token 上，只是权重不同。这是”软注意力” (Soft Attention) 名字的由来——它是连续的、可微的，不是离散的选择。

³视频画面时间区间：00:08:30–00:09:00

3.3 从相似度到权重：Softmax 的作用

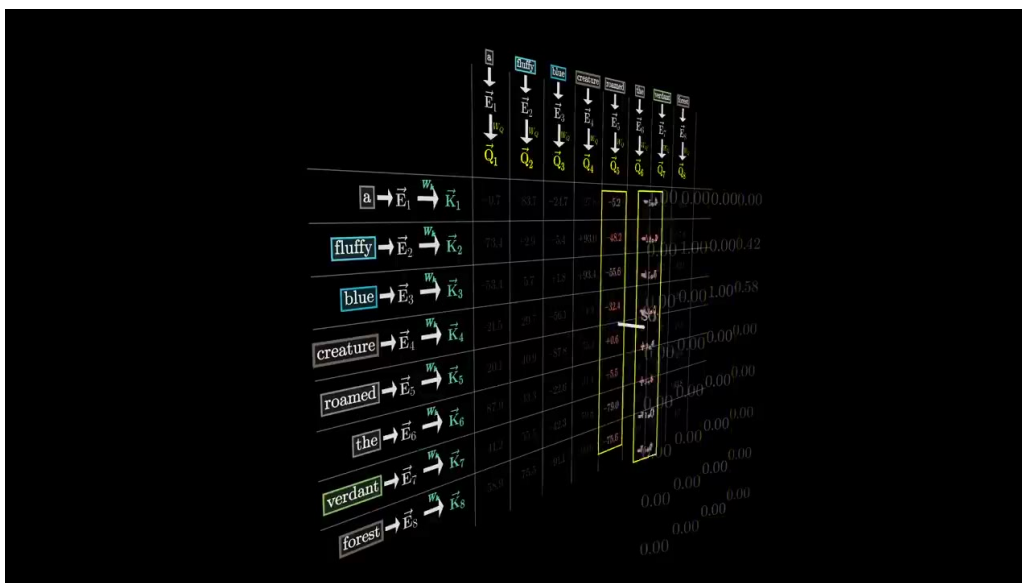


图 4: Softmax 把相似度分数转化为概率权重⁴

原始的点积得分范围很宽（可能从负几十到正几十），不能直接作为权重。Softmax 函数做两件事：

1. 把分数**指数化**，放大差距（高分更高，低分更低）
2. 把结果**归一化**，让所有权重之和为 1

结果是每个 token 获得一个“注意力分布”——一个和为 1 的概率向量，表示模型认为其他 token 与当前 token 的相关程度。

⁴ 视频画面时间区间：00:10:00–00:10:30

3.4 加权求和：最终输出

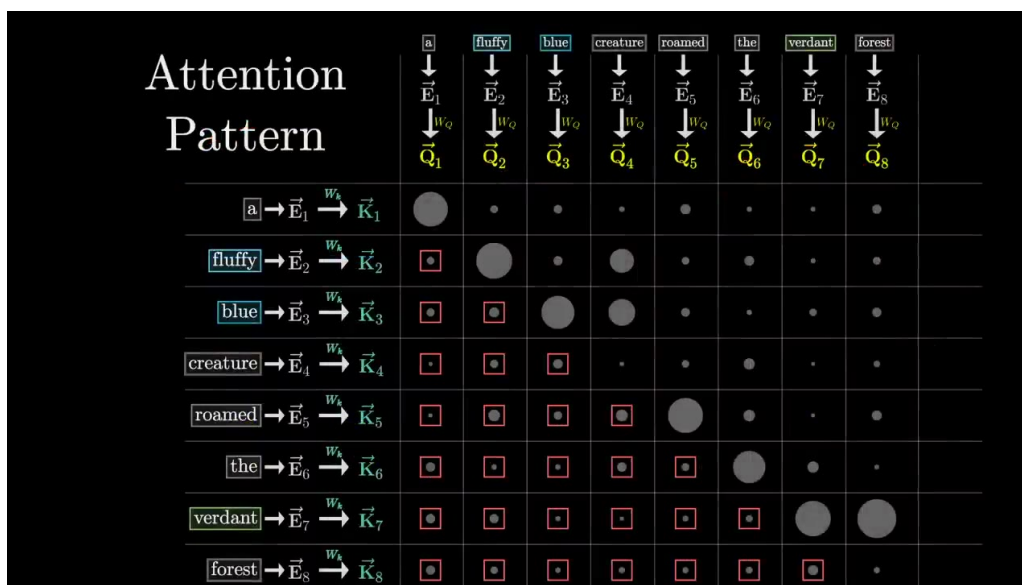


图 5: 根据注意力权重对 Value 向量做加权求和，得到上下文文化的新嵌入⁵

最后一步：用 Softmax 得到的注意力权重，对每个位置对应的 Value 向量做**加权求和**。结果是每个 token 得到一个融合了上下文信息的新向量。

为什么会改善表示？

如果没有 Attention，每个 token 的嵌入只包含它自己的信息。经过 Attention 后，“mole”的向量中不再只有它的初始嵌入，还混杂了“shrew”、“carbon dioxide”或“biopsy”的信息——这些信息来自不同的 Value 向量，按相关性加权。因此，“mole”在不同句子中的最终向量会显著不同，反映了它在上下文中的真实含义。

3.5 本章小结

Attention 的三步流程： $\text{Query} \times \text{Key} \rightarrow \text{点积得分} \rightarrow \text{Softmax 归一化} \rightarrow \text{加权求和 Value}$ 。整个流程是纯向量运算，可微可并行。

4 Query、Key、Value 矩阵：可学习的参数

4.1 从向量到矩阵

上节描述的流程涉及 Query、Key、Value 三种向量，但实际中这些向量不是直接由嵌入本身充当的。Transformer 为每个 token 使用三个不同的**线性变换矩阵** (W^Q 、 W^K 、 W^V) 来生成它们。

⁵视频画面时间区间：00:12:00-00:12:30



图 6: 三个可训练的权重矩阵分别生成 Q、K、V ⁶

具体地，对于一个输入的嵌入向量 x ：

- $q = W^Q \cdot x$ (Query 向量)
- $k = W^K \cdot x$ (Key 向量)
- $v = W^V \cdot x$ (Value 向量)

为什么需要三个不同的矩阵？

把 Q 和 K 分开，允许模型在”搜索”和”被搜索”的角色中使用不同的权重。一个词在被其他词关注时（作为 Key）和关注其他词时（作为 Query），可以是不同的投影方向。Value 则负责决定”如果被选中，我贡献什么信息”——它可以与搜索用的 Q 和 K 完全不同。这为模型提供了极大的灵活性：一句中的主语可能在”搜索”谓语时用一套权重，而被其他代词”搜索”时用另一套。

4.2 完整的 Self-Attention 公式

标准 Self-Attention 的计算公式为：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

其中：

- Q 是所有 token Query 向量的堆叠矩阵
- K 是所有 token Key 向量的堆叠矩阵
- V 是所有 token Value 向量的堆叠矩阵
- d_k 是 Key 向量的维度，用于缩放点积（防止 softmax 进入梯度饱和区）

⁶视频画面时间区间：00:16:00–00:16:30

- $\sqrt{d_k}$ 是缩放因子——这是论文中的一个重要工程细节

Scaling 为什么重要

如果不除以 $\sqrt{d_k}$, 当 d_k 很大时, 点积的方差会随维度增长而变大。大的点积值经过 Softmax 后会使梯度变得极小 (Softmax 进入”几乎 one-hot”状态), 导致训练困难。这就是为什么这个缩放因子是结构性的, 而不是可选的技巧。

4.3 本章小结

Q、K、V 不是手工设计的固定值, 而是**通过训练学习得到的投影**。三个矩阵让模型可以独立地学习”如何搜索”、”如何被搜索”和”如何提供信息”。

5 Self-Attention 如何在 Transformer 中流动

5.1 单层 Self-Attention 的完整路径

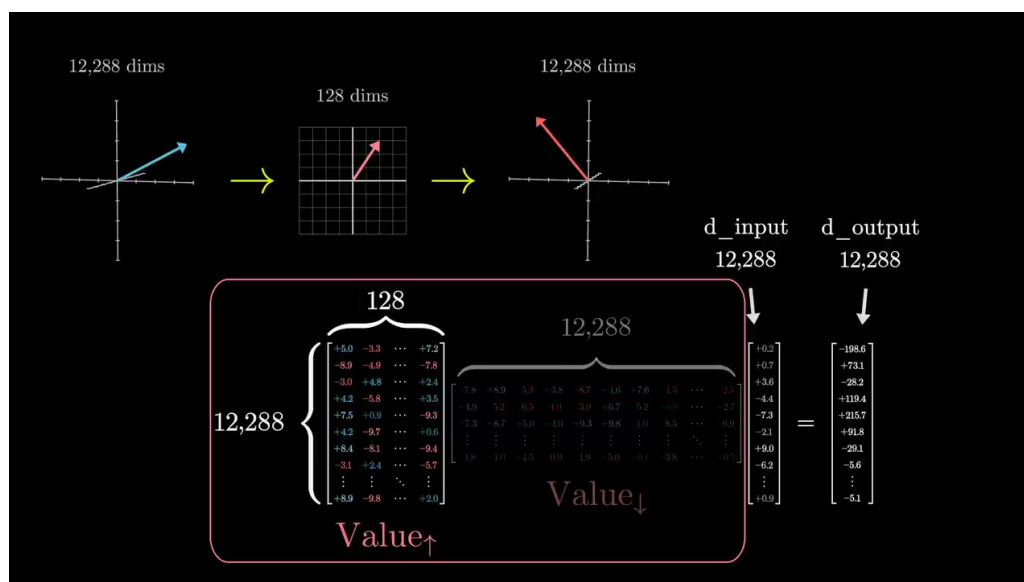


图 7: Self-Attention 层的完整数据流⁷

在 Transformer 的每一层中, Self-Attention 的执行顺序为:

1. 输入是上一层的输出嵌入 (形状: 序列长度 $\times$ 嵌入维度)
2. 通过三个线性变换分别生成 Q、K、V
3. $Q \times K^T$ 计算所有 token 两两之间的注意力得分矩阵
4. 除以 $\sqrt{d_k}$ 后做 Softmax 归一化
5. 用注意力权重对 V 做加权求和
6. 结果经过残差连接 (加上原始输入) 和层归一化

⁷视频画面时间区间: 00:18:00-00:18:30

7. 传入前馈神经网络 (FFN) 子层
8. 再次残差连接 + 层归一化, 输出到下一层

残差连接的作用

”残差连接”指每层子层的输出都要加上其输入: $\text{output} = \text{Sublayer}(\text{input}) + \text{input}$ 。这样做的好处:

- 梯度可以直接通过残差路径反向传播, 缓解深层网络的梯度消失问题
- 原始信息有一条”高速公路”直达深层, 不会被层层变换完全扭曲

5.2 本章小结

Attention 层只是 Transformer 层的一个子模块。每个 Transformer 层 = Self-Attention 子层 + FFN 子层, 两个子层各有残差连接和层归一化。深层结构让模型能逐层构建越来越抽象的上下文表示。

6 多头注意力: 并行关注不同方面

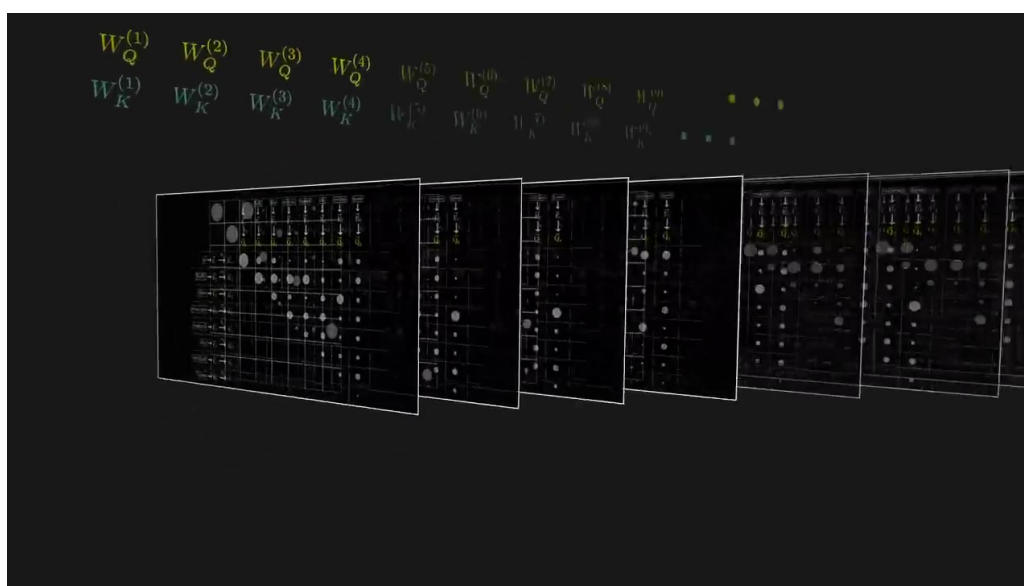


图 8: 多头注意力并行运行多个 Attention 头, 每个头关注不同的关系⁸

6.1 为什么需要多头?

单头注意力在 Q、K、V 三组矩阵的约束下, 只能学习”一种”注意力模式。但语言中的关系是多维的: 一个词可能同时需要关注它的语法主语、修饰它的形容词、以及远处的动词——这些是不同类型的”相关性”。

多头注意力 (Multi-Head Attention) 的解决方案: 并行运行 h 组不同的 Q、K、V 矩阵 (通常 $h = 8$ 或 $h = 16$), 每组称为一个”头” (head)。

⁸视频画面时间区间: 00:21:00-00:21:30

- 每个头独立学习自己的注意力模式
- 一个头可能学会关注语法关系，另一个头学会关注距离远近，再一个头学会关注语义相似性
- 所有头的输出被拼接在一起，然后通过一个输出矩阵 W^O 投影回原始维度

6.2 参数规模

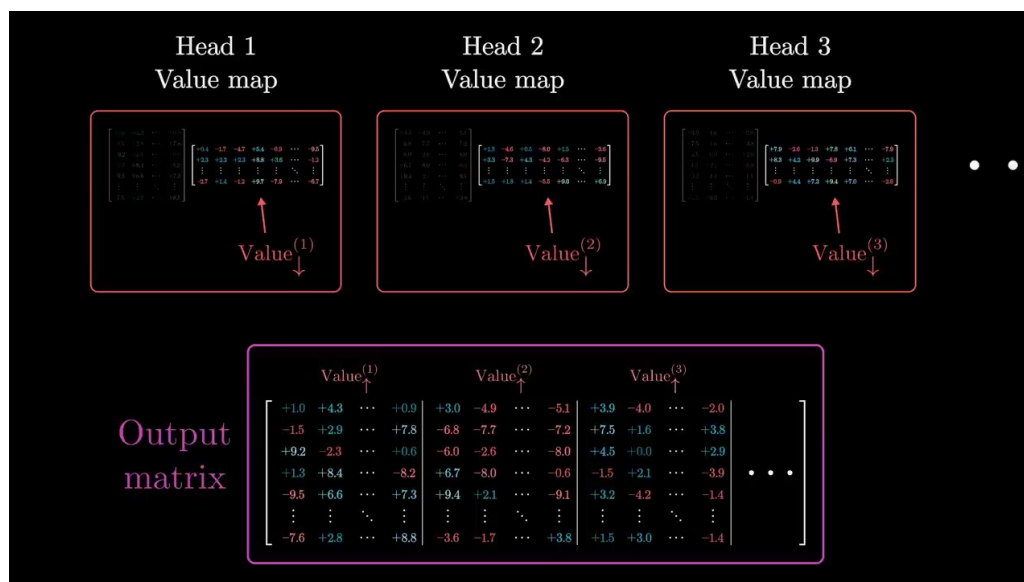


图 9: 每个注意力块包含约 6 亿参数（在 GPT-3 规模下）⁹

以 GPT-3 为例的注意力参数估算（视频中 3Blue1Brown 提供的数字）：

- 单头：Q、K、V 三组矩阵各约 7200 万参数（嵌入维度 $12288 \times$ 头维度 $128 =$ 约 160 万，乘以多个合并因子后放大）
- 96 个头：约 2.16 亿参数
- 加上输出投影矩阵：单层总注意力参数约 6 亿
- 96 层：总注意力参数约 600 亿，占 GPT-3 总参数（1750 亿）的三分之一以上

注意力的参数效率

Attention 的参数数量**不依赖于序列长度**——无论处理 10 个 token 还是 10000 个 token，Q、K、V 矩阵的大小是固定的。计算量随序列长度增长 ($O(n^2)$)，但参数数量是恒定的。这是 Attention 相比于 RNN 在长序列上的核心优势之一。

6.3 本章小结

多头注意力是 Transformer 并行处理能力的集中体现。每个头独立关注不同的语言关系，结果被拼接投影后传递到下一层。

⁹视频画面时间区间：00:23:00-00:23:30

7 总结与延伸

7.1 核心机制回顾

3Blue1Brown 在本章中从三个方面拆解了 Attention 机制：

- **为什么需要 Attention**：初始嵌入不含上下文信息，需要从其他 token 处聚合相关信息
- **Attention 如何工作**： $Q \times K$ 打分 $\rightarrow$ Softmax 归一化 $\rightarrow$ 加权求和 V
- **Attention 的扩展**：多头注意力并行关注不同关系，多层堆叠逐步构建高层次理解

7.2 视频中的关键洞察

3Blue1Brown 反复强调的是**直观理解先于公式**。Attention 的每一个数学操作（点积、Softmax、加权求和）都有清晰的几何和物理直觉：

- 点积 = 方向对齐程度（余弦相似度）
- Softmax = 软竞争（放大差距但保留全连接）
- 加权求和 = 信息混合（每个 token 都贡献一点，按重要性比例）

从本章到下一章

3Blue1Brown 预告了深度学习的后续章节：将深入 LLM 的事实存储、模型编辑等更前沿的主题。Attention 作为 Transformer **唯一**的核心创新（因此论文标题叫“Attention Is All You Need”），是理解所有后续内容的基础。

7.3 拓展阅读

- [原始论文] Attention Is All You Need (Vaswani et al., 2017)
- 3Blue1Brown 深度学习系列完整播放列表
- Christopher Olah 的 LSTM 讲解——Attention 的前身理解